Computer Networks. Unit 1: Introduction

Notes of the subject *Xarxes de Computadors, Facultat Informàtica de Barcelona, FIB*

Llorenç Cerdà-Alabern

September 7, 2026

Contents

1 Unit 1: Introduction **1**
1.1 Computer Networks Career Example . . . 1
1.2 What is a Computer Network? . . . 2
1.3 Bits per second (bps) . . . 2
1.4 Packet switching URL . . . 3
1.5 Standardization Bodies . . . 3
1.6 ISO OSI Reference Model URL . . . 3
1.7 TCP/IP Architecture URL . . . 4
1.8 Internet Infrastructure URL . . . 4
1.9 Encapsulation URL . . . 5
1.10 TCP/IP Implementation URL . . . 5
1.11 Client Server Paradigm URL . . . 6
1.12 Transport layer: UDP/TCP . . . 6
1.13 Practical examples . . . 6
1.14 List of Acronyms . . . 7

1 Unit 1: Introduction

1.1 Computer Networks Career Example

- **Information Technology (IT) Department** wikipedia
  - Enables the company's employees to communicate, collaborate, automate routine tasks, etc through **computer networks**
- **IT Functions:**
  - **Infrastructure**: Network hardware, maintenance, planning and growth
  - **Governance**: Security policies, data management, backups, contingencies
  - **Functionality**: Maintain and provide information systems (collect, process, store, and distribute information)
  - **Company Website**
- At **FIB**: Speciality in Information Technology (IT)

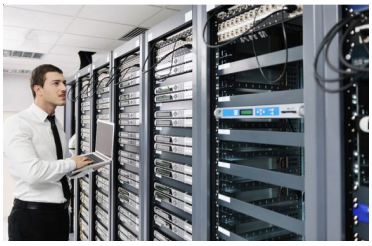


Figure 1: Data Center. Racks with servers, routers and switches.

source:https://www.marqueewfs.com/four-reasons-department-work-important

1.2 What is a Computer Network?

Brief history:

- **1830** Telegraph
- **1875** Alexander Graham Bell patent the telephone
- **1951** First commercial computer
- **1960** ARPANET, BSD, Public networks rediris geant
- **1972** First commercial Packet Switching Network, X.25
- **1990** The Internet is opened to the general public

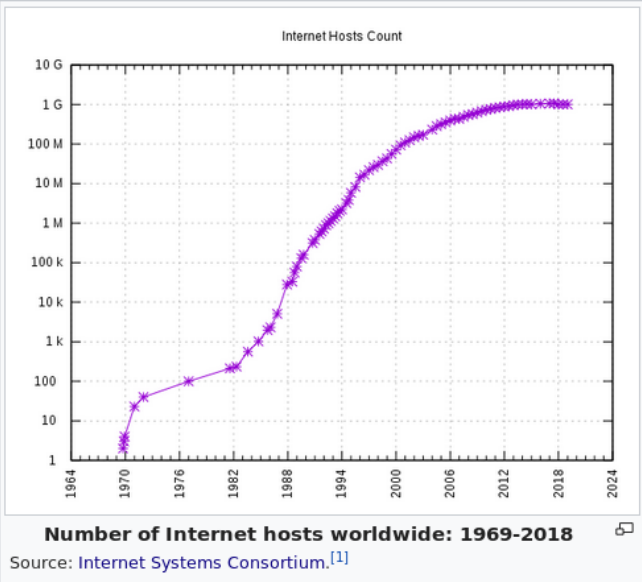
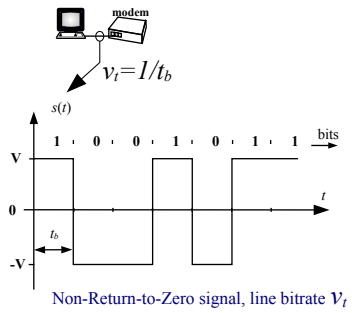


Figure 2: Number of hosts in the Internet: less than 10 in 1970 to 10^9 in 2018.

1.3 Bits per second (bps)

- **line bitrate**



SI Prefixes:

k	kilo: 10^3
M	Mega: 10^6
G	Giga: 10^9
T	Tera: 10^{12}
P	Peta: 10^{15}

Figure 3: Transmission signal: positive voltage is the bit 1, negative voltage is the bit 0.

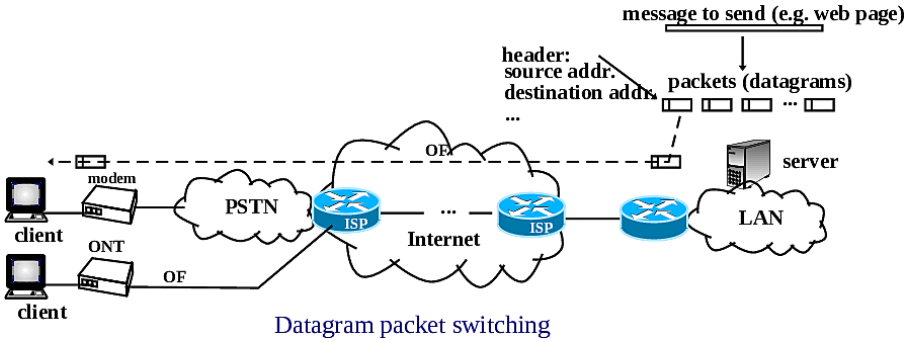
- **throughput** (velocidad efectiva)

$$v_{ef}[\text{bps}] = \frac{\text{number of information bits}}{\text{observation time}}$$

- Practical example: throughput with speedtest

1.4 Packet switching URL

- **Virtual Circuit**: Connection oriented, used in **WANs**, e.g. X.25, Frame Relay, ATM. We will not study them
- **Datagram**: Connectionless, used in the **Internet**
- Packets are sent using Store and forward



Datagram packet switching

Figure 4: In datagram packet switching addresses identify a unique destination. At datagram arrival routers decide best next router to forward the datagram to reach the destination.

Acronyms:

ONT:	Optical Network Terminal
OF:	Optical Fiber
PSTN:	Public Switched Telephone network
WAN:	Wide Area Network
LAN:	Local Area Network
ATM:	Asynchronous Transfer Mode

1.5 Standardization Bodies

1. *Int. Telecommunication Union*, **ITU**
   - WAN standards. URL
2. *Int. Organization for Standardization*, **ISO**
   - Industrial standards. URL.
3. *Institute of Electrical and Electronics Engineers*, **IEEE**
   - LAN standards. URL.
4. *Telecommunications Industry Association*, **TIA**
   - Cabling standards. URL.
5. *World Wide Web Consortium*, **W3C**. URL

Internet:

1. *Internet Engineering Task Force*, **IETF**. URL.
   - Request For Comments, **RFCs**. URL

1.6 ISO OSI Reference Model URL

OSI: *Open Systems Interconnection*

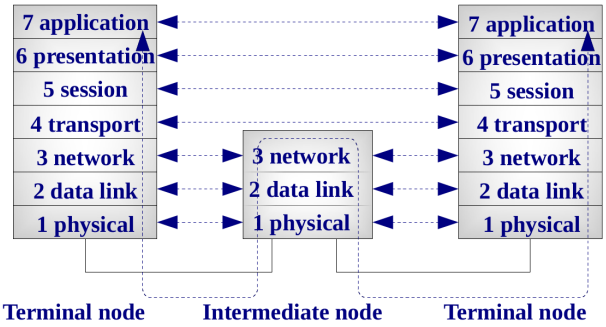


Figure 5: OSI reference model. Terminal nodes have layers 1 physical, 2 data-link, 3 network, 4 transport, 5 session, 6 presentation, 7 application. Intermediate nodes have layers 1-2-3.

Definitions

- **Protocols** enable an entity to interact with a corresponding entity at the same layer.
- At each layer entities exchange protocol data units, **PDUs**.
- PDU contains a **payload** and protocol-related **header**.

Layers

1. **Physical**: transmission of bits over a physical transmission medium.
2. **Data Link**: allows the transmission of PDUs over the physical. Framing and error detection.
3. **Network**: route packets.
4. **Transport**: reliability, flow control and segmentation.
5. **Session**: analogous to a UNIX user session.
6. **Presentation**: data representation of images, floating point numbers, etc.
7. **Application**: applications using the network.

1.7 TCP/IP Architecture URL

- No RFC specifies the TCP/IP arquitecture.

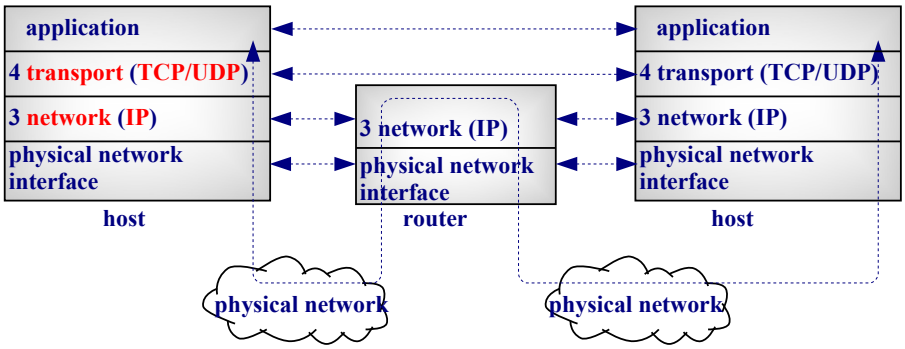


Figure 6: OSI layers in TCP/IP: layers 3 and 4 correspond to IP and TCP/UDP, respectively.

1.8 Internet Infrastructure URL

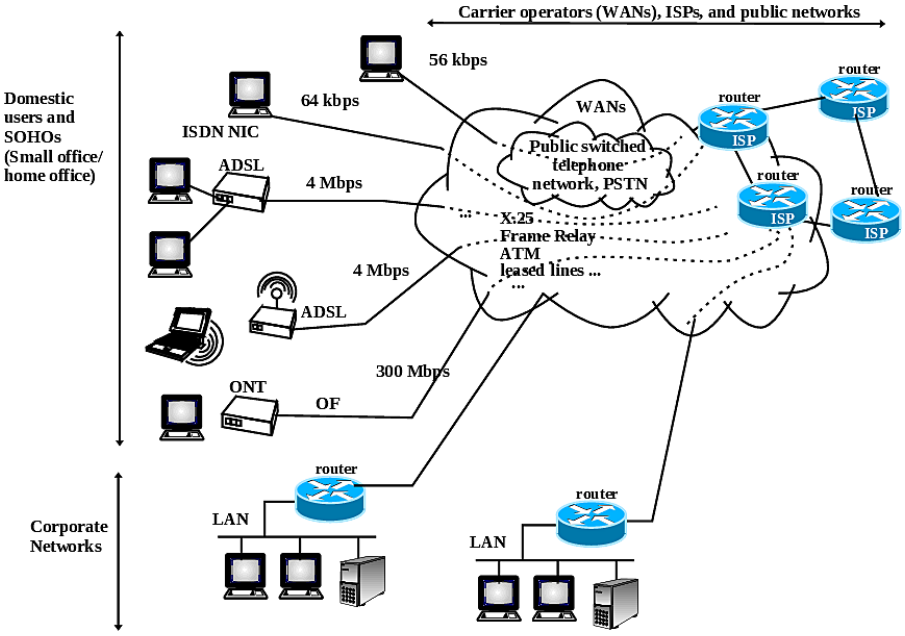


Figure 7: The Internet is an interconnection of hosts and routers that exchange IP datagrams. Premises networks (LANs) are connected to ISPs, which use network carriers WANs for global connectivity.

Acronyms:

ADSL:	Asymmetric Digital Subscriber Line
ATM	Asynchronous Transfer Mode
ISDN:	Integrated Services Digital Network
ISP:	Internet Service Provider
LAN:	Local Area Network
NIC	Network Interface Card
OF:	Optical Fiber
ONT:	Optical Network Terminal
PSTN:	Public Switched Telephone network
WAN:	Wide Area Network

1.9 Encapsulation URL

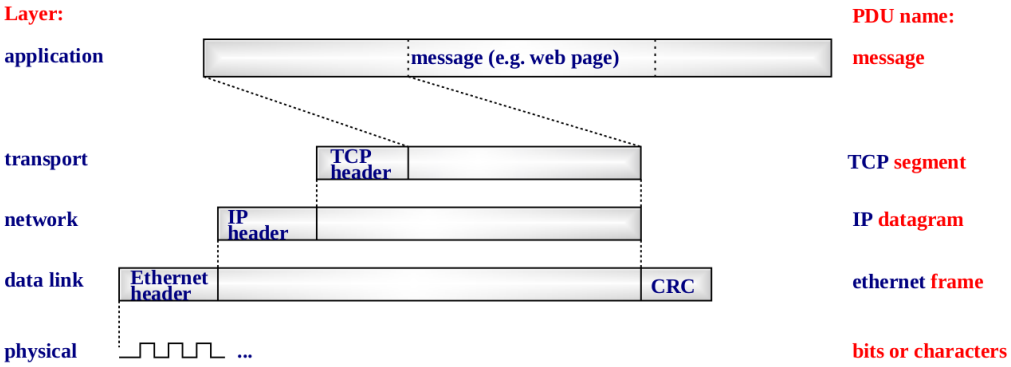


Figure 8: Each layer encapsulates an upper layer PDU adding a header with the information required by the protocol. Transport, network and data-link PDUs in TCP/IP are TCP-segments, IP datagrams and frames, respectively.

Acronyms:

PDU:	Protocol Data Unit
CRC:	Cyclic Redundancy Check

Network sniffers (bash)

```
sudo tcpdump -ni wlan0 # command line sniffer
sudo wireshark         # graphical sniffer
```

Figure 9: Practical example: capturing traffic with tcpdump and wireshark.

1.10 TCP/IP Implementation URL

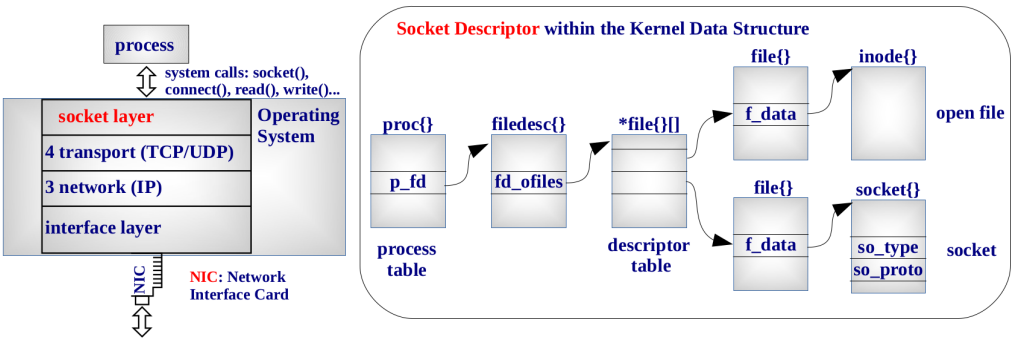


Figure 10: TCP/IP protocols are implemented in the operating system kernel. Networking data structures are related with the file system: applications using the network read/write sockets instead of files.

TCP and UDP sockets (bash)

```
netstat -nt # list TCP sockets
netstat -nu # list UDP sockets
```

Figure 11: Practical example: list of TCP and UDP sockets.

Sockets opened by a browser (bash)

```
netstat -nt
```

Figure 12: Practical example: list the TCP sockets opened by a web browser.

1.11 Client Server Paradigm URL

- The server "listens" a **well known port** (< 1024).
- The client connects with an **ephemeral port** (>=1024).
- Transport layer protocols are **TCP/UDP**. They send the source and destination ports in the PDU header.

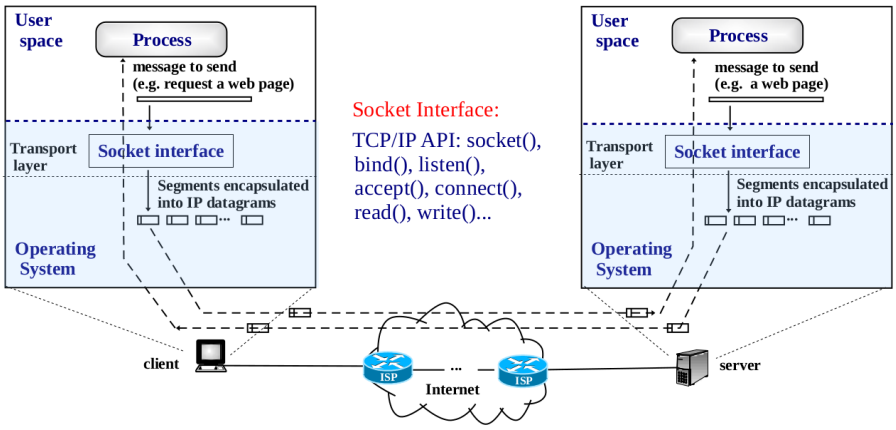


Figure 13: Client Server Paradigm: Sockets opened by the processes are identified by *port numbers*. Client-socket ports are assigned by the operating system. They are larger than 1023 and are called *ephemeral*. Server-socket ports are assigned by the application and are called *well-known* because they are standardized and identify the service. Source and destination port numbers are sent in the TCP/UDP headers.

Acronyms:
API: Application Programming Interface

TCP and UDP servers (bash)

```
netstat -nat        # list all TCP sockets (client and server)
netstat -nau        # list all UDP sockets (client and server)
file /etc/services  # well known ports
```

Figure 14: Practical example: list server-sockets in the host and list the well-known ports in the UNIX /etc/services file.

1.12 Transport layer: UDP/TCP

- UDP **User Datagram Protocol**: Connectionless, no reliable.
- TCP **Transmission Control Protocol**: Connection oriented, reliable.

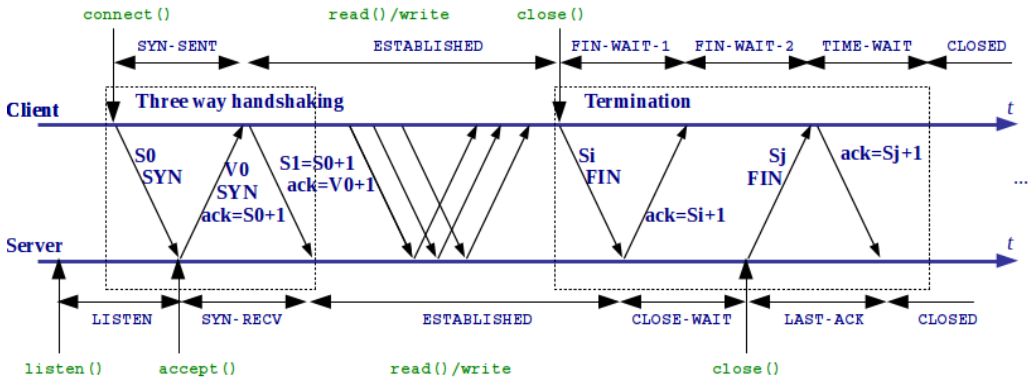


Figure 15: Time diagram with the connection establishment and termination of a TCP connection.

1.13 Practical examples

Capturing UDP/TCP packets with wireshark (bash)

```
wireshark
```

Figure 16: Practical example: Use wireshark to capture the packets exchanged by a client/server using UDP and TCP.

Minimal UDP server (ruby)

```ruby
#!/usr/bin/ruby -w
# -*- eval: (text-scale-adjust 3) -*-
require 'socket'

server = UDPSocket.new # Server
server.bind("0.0.0.0", 2000) # bind to port 2000
puts server.recvfrom(1000) # read message (max 1000 bytes)
server.close # close socket
```

Minimal UDP client (ruby)

```ruby
#!/usr/bin/ruby -w
# -*- eval: (text-scale-adjust 3) -*-
require 'socket'

server = UDPSocket.new
server.connect("127.0.0.1", 2000)
server.send("Hello world", 0) # send message
```

Minimal TCP server (ruby)

```ruby
#!/usr/bin/ruby -w
# -*- eval: (text-scale-adjust 3) -*-
require 'socket'

server = TCPServer.new 2000 # bind to port 2000
client = server.accept # Wait for a client to connect
puts client.gets # read message
client.close # close socket
```

Minimal TCP client (ruby)

```ruby
#!/usr/bin/ruby -w
# -*- eval: (text-scale-adjust 3) -*-
require 'socket'

server = TCPSocket.new('127.0.0.1', 2000)
server.puts "Hello world" # send message
server.close # close socket
```

Additional info: implementation of a **concurrent server**

- **TCP** in C:
  - https://man7.org/linux/man-pages/man2/accept.2.html
    accept system call creates a new socket in CONNECTED state. The original socket is unaffected.
  - https://jameshfisher.com/2017/02/25/tcp-server-fork
    Uses the **fork** system call to spawn a new child process for each accepted connection.
- **UDP** is stateless: the application is responsible to implement concurrent connections

1.14 List of Acronyms

API	Application Programming Interface
ATM	Asynchronous Transfer Mode
CRC	Cyclic Redundancy Check
IP	Internet Protocol
ISDN:	Integrated Services Digital Network
ISP:	Internet Service Provider
IT	Information Technology
LAN	Local Area Network
NIC	Network Interface Card
OF:	Optical Fiber
ONT:	Optical Network Terminal
PDU	Protocol Data Unit
PSTN	Public Switched Telephone Network
RFC	Request For Comments
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
URL	Uniform Resource Locator
WAN	Wide Area Network